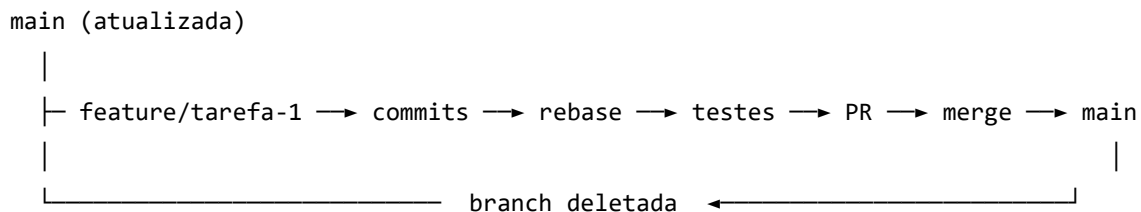


Guia prático: fluxo de trabalho com Git e GitHub

Este guia segue o modelo **GitHub Flow**: uma branch por tarefa, Pull Request e merge na `main`. É o fluxo mais usado no dia a dia de times pequenos e médios (diferente do **Git Flow** "oficial", que usa branches extras como `develop`, `release/*` e `hotfix/*`, indicado para projetos com releases mais formais).

A ideia central é simples: **a `main` deve estar sempre estável**. Todo trabalho novo acontece em uma branch separada, é revisado via **PR**, e só depois entra na `main`.

Visão geral do ciclo



Cada tarefa passa por: **atualizar** → **criar branch** → **codar** → **commitar** → **sincronizar** → **testar** → **enviar** → **revisar** → **mergear** → **limpar**.

Passo a passo

1. Atualizar a main

```
git switch main
git pull origin main
```

Troca para a branch principal e baixa as últimas alterações do remoto. Começar uma tarefa a partir de uma `main` desatualizada gera conflitos evitáveis depois.

2. Criar a branch da tarefa

```
git switch -c feature/minha-tarefa
```

Cria uma branch nova a partir da `main` atual e já muda para ela. Isso mantém a `main` sempre estável enquanto você trabalha isolado.

Convenção de nomes:

Prefixo	Uso
feature/	nova funcionalidade
fix/	correção de bug
chore/	manutenção, configs
test/	testes

Ex: `feature/login-google`, `fix/timeout-checkout`

3. Trabalhar no código

Edite os arquivos normalmente, fazendo **commits pequenos e frequentes** em vez de um commit gigante no fim do dia. Isso facilita revisão, leitura do histórico e reversão se algo der errado.

4. Conferir o que mudou

```
git status
git diff
```

`status` lista arquivos alterados/criados/deletados; `diff` mostra linha a linha o que mudou. Revisar antes de commitar evita subir arquivos por engano (logs, `.env`, arquivos de debug).

5. Preparar o commit (staging)

```
git add -p
```

Modo interativo: mostra cada trecho alterado e pergunta se deve entrar no commit. É mais seguro que `git add .`, que adiciona tudo sem revisão.

Alternativa simples:

```
git add caminho/do/arquivo.js
```

6. Commitar

```
git commit -m "feat: minha alteração"
```

Salva o que está em staging como um novo ponto no histórico. Use **Conventional Commits** para manter o histórico legível:

Tipo	Uso
feat	nova funcionalidade
fix	correção de bug
docs	documentação
test	testes
refactor	refatoração sem mudar comportamento
chore	tarefas de manutenção

7. Sincronizar com a main (se a branch já existe há um tempo)

```
git switch main && git pull origin main
git switch feature/minha-tarefa && git rebase main
```

Traz as mudanças recentes da `main` para a sua branch, reaplicando seus commits por cima. Evita um conflito gigante só na hora do PR — é exatamente o que aconteceu na sua tela ("1 commit behind main").

Se der conflito durante o rebase, resolva manualmente e continue:

```
git add arquivo-corrigido.js
git rebase --continue
```

8. Rodar os testes

```
npx playwright test
```

Garante que sua alteração não quebrou nada antes de pedir revisão — poupa idas e vindas no PR.

9. Enviar a branch para o GitHub

```
git push -u origin feature/minha-tarefa
```

O `-u` vincula a branch local à remota; depois disso basta `git push`.

10. Abrir o Pull Request

No GitHub, abra um PR de `feature/minha-tarefa` → `main`. É o momento de revisão (por outra pessoa ou por você mesmo) e onde rodam os testes automatizados (CI) antes do merge.

11. Merge

Depois de aprovado, faça o merge pela interface do GitHub. Estratégias disponíveis:

Estratégia	Efeito
Merge commit	mantém todos os commits + cria um commit de merge
Squash and merge	junta todos os commits da branch em um só
Rebase and merge	aplica os commits linearmente, sem commit de merge

Vale escolher uma como padrão do time.

12. Limpar a branch

```
git branch -d feature/minha-tarefa
git push origin --delete feature/minha-tarefa
```

Remove a branch local e a remota. Branches mergeadas acumulam e poluem o repositório — dá para automatizar isso marcando "Automatically delete head branches" nas configurações do GitHub.

13. Atualizar a main local

```
git switch main
git pull origin main
```

Traz o merge recém-feito para sua cópia local, deixando tudo pronto para a próxima tarefa.

☐ **Repetir**

Volte ao passo 1 para a próxima tarefa.